



Project Type: Major Project — Full Stack Web Application

Technology: MERN Stack (MongoDB, Express.js, React.js, Node.js)

Build Tool: Vite | **Styling:** Custom CSS + Bootstrap

Full Stack • REST API • Authentication • CRUD Operations



Table of Contents

- 01** Introduction & Project Overview
- 02** Objectives of the Project
- 03** Technology Stack
- 04** System Architecture
- 05** Key Features
- 06** Backend Implementation
- 07** Database Models & Schema Design
- 08** API Endpoints & Routes
- 09** Authentication & Security
- 10** Frontend Implementation
- 11** Pages & Components
- 12** Data Flow & State Management
- 13** UI/UX Design
- 14** Testing & Validation
- 15** Future Enhancements
- 16** Conclusion

1. Introduction & Project Overview

Wanderlust is a full-stack web application inspired by Airbnb that enables users to browse, create, edit, and manage property listings for travel accommodations worldwide.

The **Wanderlust** project (also branded as "**Explore**" in the UI) is a comprehensive full-stack web application that replicates the core functionality of modern property listing platforms like Airbnb. It provides a rich, interactive user experience for discovering travel stays across the globe, complete with user authentication, CRUD operations on property listings, and a review system.

The application follows a **client-server architecture** with a clear separation between the frontend (React.js single-page application) and the backend (Express.js REST API). Data is persisted in a **MongoDB** database, making this a complete **MERN Stack** application — one of the most popular and industry-standard technology stacks for modern web development.

✦ Project Scope

The application ships with **28 pre-seeded sample listings** spanning locations across the USA, Europe, Asia, Africa, and Oceania — allowing immediate demonstration of all features without manual data entry.

2. Objectives of the Project

The primary objectives of the Wanderlust project are as follows:

- 1. Build a Full-Stack Application** — Demonstrate proficiency in developing both client-side and server-side components of a modern web application using the MERN stack.
- 2. Implement RESTful API Design** — Design and implement a well-structured REST API following industry best practices for CRUD operations on property listings.
- 3. User Authentication & Authorization** — Implement a secure authentication system using Passport.js with session-based authentication and MongoDB session store.
- 4. Responsive & Modern UI** — Create an aesthetically pleasing, responsive user interface inspired by Airbnb's design language, using React.js with custom CSS styling.
- 5. Database Design** — Design and implement efficient MongoDB schemas with proper relationships (references) between collections (Listings, Reviews, Users).
- 6. Input Validation** — Implement robust server-side validation using Joi schemas to ensure data integrity.
- 7. Image Handling** — Support image uploads via drag-and-drop with Base64 encoding and URL-based image references.
- 8. Search & Filter Functionality** — Implement client-side search and price-range filtering to help users discover relevant listings.

3. Technology Stack

The project leverages the MERN stack — a proven, full-JavaScript ecosystem for building modern web applications.

3.1 Frontend Technologies

React.js 18

Component-based UI library for building interactive single-page applications with virtual DOM and hooks.

Vite 8

Next-generation build tool providing instant HMR (Hot Module Replacement) and optimized production builds.

React Router 7

Declarative client-side routing for navigation between pages without full page reloads.

Axios

Promise-based HTTP client for making API requests to the backend server.

Bootstrap 5

CSS framework providing responsive grid system and base component styles.

Custom CSS

Hand-crafted CSS with modern design patterns — glassmorphism, animations, and responsive layouts.

3.2 Backend Technologies

Node.js

JavaScript runtime built on Chrome's V8 engine, enabling server-side JavaScript execution.

Express.js 5

Minimal, flexible web framework for building robust REST APIs with middleware support.

MongoDB

NoSQL document database providing flexible schema design and horizontal scalability.

Mongoose 9

Elegant MongoDB ODM providing schema validation, middleware, and query building.

Passport.js

Authentication middleware with Local Strategy for username/password login.

Joi

Schema-based validation library for ensuring data integrity on API inputs.

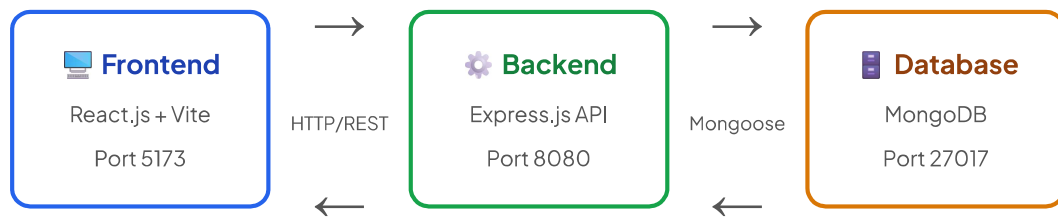
3.3 Complete Dependency Map

PACKAGE	VERSION	PURPOSE	LAYER
react	18.3.1	UI component library	Frontend
react-dom	18.3.1	DOM rendering for React	Frontend
react-router-dom	7.15.0	Client-side routing	Frontend
axios	1.7.0	HTTP client	Frontend
bootstrap	5.3.0	CSS framework	Frontend
vite	8.0.10	Build tool & dev server	Frontend
express	5.2.1	Web framework	Backend
mongoose	9.2.4	MongoDB ODM	Backend
passport	0.7.0	Authentication	Backend
passport-local	1.0.0	Local auth strategy	Backend
passport-local-mongoose	9.1.0	Mongoose-Passport integration	Backend
express-session	1.19.0	Session management	Backend
connect-mongo	6.0.0	MongoDB session store	Backend
joi	18.1.2	Input validation	Backend
cors	2.8.6	Cross-origin requests	Backend
method-override	3.0.0	HTTP method support	Backend

4. System Architecture

The application follows a three-tier client-server architecture with a clear separation of concerns between the presentation, business logic, and data layers.

High-Level Architecture Diagram



4.1 Frontend Layer (Client)

The frontend is a **React.js Single Page Application (SPA)** built with **Vite** as the build tool. It runs on port **5173** during development and communicates with the backend via HTTP requests using **Axios** and the native **Fetch API**. Client-side routing is handled by **React Router v7**, enabling seamless navigation without full page reloads.

4.2 Backend Layer (Server)

The backend is an **Express.js REST API** running on port **8080**. It handles all business logic including CRUD operations on listings, user authentication (via Passport.js), input validation (via Joi), and session management (via express-session with MongoDB store). **CORS** is configured to allow requests from the frontend origin.

4.3 Data Layer (Database)

MongoDB serves as the database, running on the default port **27017**. The database is named **wanderlust** and contains three main collections: **Listings**, **Reviews**, and **Users**. **Mongoose** is used as the ODM (Object Data Modeling) library, providing schema definitions, validation, and query building.

Project Directory Structure

```
wanderlust/
├─ backend/
│   ├── app.js           ← Main Express server entry point
│   ├── middleware.js    ← Validation & auth middleware
│   ├── schema.js        ← Joi validation schemas
│   └─ models/
│       ├── listing.js   ← Listing Mongoose model
│       ├── review.js    ← Review Mongoose model
│       └─ user.js       ← User Mongoose model
│   └─ routes/
│       ├── listing.js   ← Listing CRUD routes
│       └─ user.js       ← Auth routes (login/register/logout)
│   └─ utils/
│       ├── ExpressError.js ← Custom error class
│       └─ wrapAsync.js   ← Async error wrapper
│   └─ init/
│       ├── data.js      ← 28 sample listings seed data
│       └─ index.js      ← Database seeder script
└─ frontend/
    ├── index.html       ← HTML entry point
    ├── vite.config.js   ← Vite configuration
    └─ src/
        ├── main.jsx     ← React entry point
        ├── App.jsx      ← Root component with routing
        └─ pages/
            ├── Home.jsx  ← Home page
            ├── LoginPage.jsx ← Login form
            ├── RegisterPage.jsx ← Registration form
            ├── CreateListing.jsx ← New listing form
            ├── EditListing.jsx ← Edit listing form
            └─ ShowListing.jsx ← Listing detail page
        └─ components/
            ├── Navbar.jsx ← Navigation bar
            ├── Listings.jsx ← Listings grid + categories
            ├── HeroBanner.jsx ← Hero section
            ├── ImageUpload.jsx ← Drag & drop image upload
            └─ SkeletonGrid.jsx ← Loading skeleton UI
        └─ styles/
            ├── Hero.css, ListingForm.css, LoginPage.css,
            ├── RegisterPage.css, ShowListing.css
            └─ (+ component-level CSS files)
```

5. Key Features



Property Listings CRUD

Create, read, update, and delete property listings with title, description, price, location, country, and images.



User Authentication

Secure session-based authentication with registration, login, logout using Passport.js Local Strategy.



Search & Filter

Real-time search across title, location, and country. Price range filter with min/max sliders.



Image Upload

Drag-and-drop image upload with preview, or paste an image URL. Supports PNG, JPG, WEBP formats.



Star Ratings

Visual 5-star rating system with SVG-rendered stars and numeric rating display on listing cards.



Review System

Users can post reviews with comments and ratings (1–5 stars) on individual listings.



Category Navigation

14 browsing categories (Trending, Mountains, Castles, Beachfront, etc.) with Airbnb-style icon bar.



Responsive Design

Fully responsive layout adapting to desktop, tablet, and mobile screens with grid breakpoints.



Skeleton Loading



Tax Toggle

Shimmer-animated skeleton cards shown while data loads, providing smooth UX transitions.

"Display total after taxes" toggle showing prices with or without 18% GST calculation.



Delete Confirmation

Animated modal confirmation dialog before deleting a listing, preventing accidental deletions.



Input Validation

Server-side Joi validation ensures all required fields are present and properly formatted.

6. Backend Implementation

The backend is built with Express.js 5 and serves as a RESTful API that the React frontend communicates with over HTTP.

6.1 Server Configuration (app.js)

The main server file configures the Express application with the following middleware stack:

```
// Core Middleware Stack
const express = require("express");
const app = express();

// Body parsing
app.use(express.urlencoded({ extended: true, limit: "10mb" }));
app.use(express.json({ limit: "10mb" }));

// CORS - Allow frontend at localhost:5173
app.use(cors({
  origin: "http://localhost:5173",
  credentials: true,
  methods: ["GET", "POST", "PUT", "DELETE"],
}));

// Session with MongoDB Store (7-day expiry)
app.use(session({
  store: new MongoStore({ mongoUrl: MONGO_URL }),
  secret: "thisshouldbeabettersecret",
  cookie: { maxAge: 1000 * 60 * 60 * 24 * 7 }
}));

// Passport Authentication
app.use(passport.initialize());
app.use(passport.session());

// Server listens on port 8080
app.listen(8080);
```

6.2 Middleware Functions

MIDDLEWARE	FILE	PURPOSE
<code>validateListing</code>	<code>middleware.js</code>	Validates listing data against Joi schema; returns 400 error with details on failure
<code>validateReview</code>	<code>middleware.js</code>	Validates review data (rating 1–5, required comment) against Joi schema
<code>isLoggedIn</code>	<code>middleware.js</code>	Checks <code>req.isAuthenticated()</code> ; returns 401 JSON if user is not logged in
<code>wrapAsync</code>	<code>utils/wrapAsync.js</code>	Wraps async route handlers to catch promise rejections and pass to error handler

6.3 Error Handling

The application uses a custom `ExpressError` class that extends the native JavaScript `Error` class with an additional `statusCode` property. The global error handler catches all errors and responds with the appropriate HTTP status code and message.

```
class ExpressError extends Error {
  constructor(statusCode, message) {
    super();
    this.statusCode = statusCode;
    this.message = message;
  }
}
```

7. Database Models & Schema Design

The application uses three Mongoose models that represent the core entities: Listings, Reviews, and Users. These are stored in the MongoDB database named "wanderlust".

7.1 Listing Model

Collection: listings

```
title String — required
description String — optional
image Object — { url: String (default: Unsplash), filename: String }
price Number — listing price per night
location String — city/area name
country String — country name
reviews [ObjectId] — ref → "Review" collection
```

7.2 Review Model

Collection: reviews

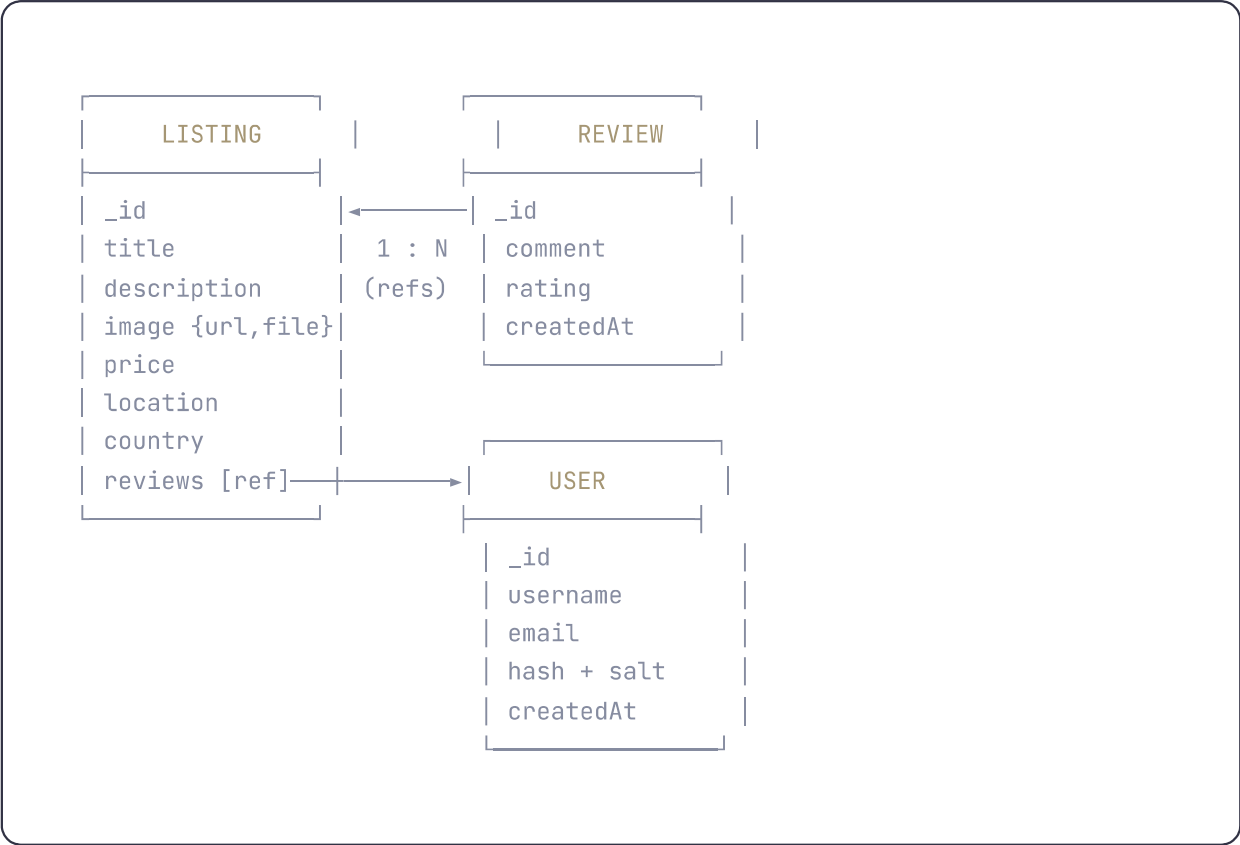
```
comment String — review text
rating Number — min: 1, max: 5
createdAt Date — default: Date.now()
```

7.3 User Model

Collection: users

```
email String — required, unique
createdAt Date — default: Date.now
username String — auto-added by passport-local-mongoose
hash String — hashed password (auto by plugin)
salt String — password salt (auto by plugin)
```

Entity Relationship Diagram



8. API Endpoints & Routes

The backend exposes a RESTful API with clearly defined endpoints for listings, reviews, and user authentication.

8.1 Listing Routes — `/listings`

METHOD	ENDPOINT	DESCRIPTION	AUTH REQUIRED
GET	<code>/listings</code>	Fetch all listings (index)	No
GET	<code>/listings/:id</code>	Fetch a single listing by ID	No
POST	<code>/listings</code>	Create a new listing	No*
PUT	<code>/listings/:id</code>	Update an existing listing	No*
DELETE	<code>/listings/:id</code>	Delete a listing	No*

8.2 Review Routes — `/listings/:id/reviews`

METHOD	ENDPOINT	DESCRIPTION	VALIDATION
POST	<code>/listings/:id/reviews</code>	Create a new review for a listing	validateReview (Joi)

8.3 User/Auth Routes — /users

METHOD	ENDPOINT	DESCRIPTION	RESPONSE
POST	/users/register	Register a new user account	201 + user JSON
POST	/users/login	Authenticate user (Passport Local)	200 + user JSON
POST	/users/logout	End user session	200 + success message
GET	/users/current-user	Get currently authenticated user	User object or null

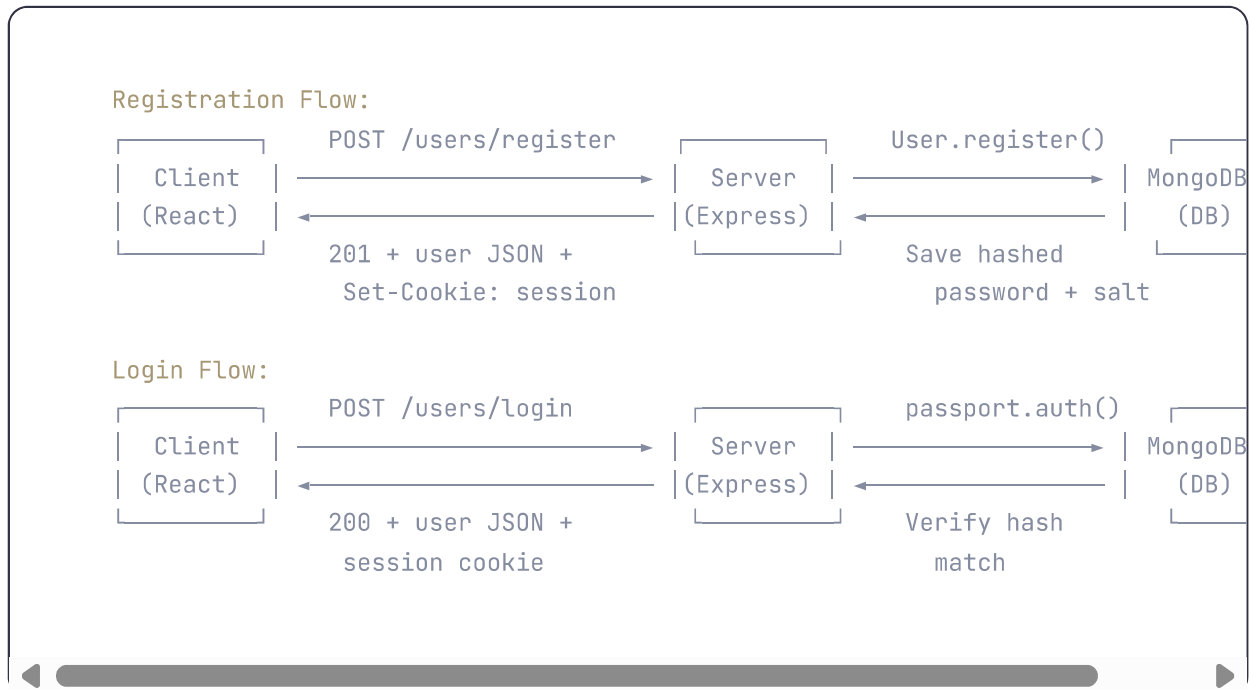
8.4 Joi Validation Schemas

```
// Listing Validation
const listingSchema = Joi.object({
  listing: Joi.object({
    title: Joi.string().required(),
    description: Joi.string().required(),
    location: Joi.string().required(),
    country: Joi.string().required(),
    price: Joi.number().required().min(0),
    image: Joi.object({ url: Joi.string().allow("") })
  }).required()
});

// Review Validation
const reviewSchema = Joi.object({
  review: Joi.object({
    rating: Joi.number().required().min(1).max(5),
    comment: Joi.string().required()
  }).required()
});
```

9. Authentication & Security

9.1 Authentication Flow



9.2 Session Management

- **Session Store:** MongoDB via `connect-mongo` — sessions persist across server restarts
- **Session Expiry:** 7 days (configurable via `maxAge`)
- **Cookie Settings:** `httpOnly: true` , `sameSite: "lax"` — prevents XSS access to cookies
- **Lazy Updates:** `touchAfter: 24 * 3600` — reduces unnecessary session writes

9.3 Password Security

Passwords are **never stored in plaintext**. The `passport-local-mongoose` plugin automatically handles password hashing with **PBKDF2** algorithm, generating a unique salt for each user. The plugin adds `hash` and `salt` fields to the User model and provides `authenticate()` , `register()` , `serializeUser()` , and `deserializeUser()` methods.

10. Frontend Implementation

The frontend is a React.js 18 single-page application built with Vite, featuring component-based architecture, client-side routing, and modern CSS styling.

10.1 Application Entry Point

The React application mounts at `#root` inside `index.html`. The `main.jsx` file renders the `<App />` component within `React.StrictMode` and imports Bootstrap CSS globally.

10.2 Routing Architecture

The application uses **React Router v7** with `BrowserRouter` for client-side routing. The `<Navbar />` component is rendered outside of `<Routes>`, making it visible across all pages.

ROUTE PATH	COMPONENT	PURPOSE
<code>/</code>	Home	Home page — renders the Listings component
<code>/listings</code>	Listings	All listings grid with categories, search, and filter
<code>/listings/:id</code>	ShowListing	Individual listing detail page
<code>/create</code>	CreateListing	Form to create a new listing
<code>/edit/:id</code>	EditListing	Form to edit an existing listing
<code>/login</code>	LoginPage	User login form
<code>/register</code>	RegisterPage	User registration form

10.3 Design System

- Font:** Plus Jakarta Sans (Google Fonts) — modern, geometric sans-serif
- Primary Brand Color:** `#FF385C` (Airbnb-inspired pink/red)
- Text Color:** `#222222` on `#ffffff` background
- Card Radius:** 14px rounded corners for card elements
- Hover Effects:** Shadow elevation + subtle image zoom on card hover
- Animations:** Shimmer loading, fadeIn/slideUp modals, background scaling

11. Pages & Components

11.1 Navbar Component

A **sticky navigation bar** (z-index: 200, height: 70px) that appears on all pages. Features a brand logo with "Explore" text, a center search bar that redirects to `/listings?search=...`, and conditional auth links (Sign Up / Log In when not authenticated, Welcome message + Logout when authenticated).

11.2 Listings Component (Main Grid)

The heart of the application — a **273-line component** that displays all property listings in a responsive grid layout. Key sub-features include:

- **Category Bar:** 14 Airbnb-style category buttons (Trending, Rooms, Mountains, Castles, Beachfront, etc.) with custom SVG icons, horizontally scrollable and sticky below the navbar.
- **Tax Toggle:** A pill-shaped switch to display prices with or without 18% GST.
- **Filter Sidebar:** Modal overlay with dual price range sliders (min/max), displaying prices in ₹ (Indian Rupees).
- **Star Ratings:** Deterministic fake ratings (3.5–5.0) generated from listing ID for visual appeal.
- **Listing Cards:** Image, title, star rating, location, and price per night — each linking to the detail page.
- **Skeleton Loading:** Shimmer-animated placeholder cards while data loads.

11.3 ShowListing Page

Displays detailed information for a single listing including a hero image (16:9 ratio), title, location with pin icon, description, and price formatted in ₹ (Indian locale). Includes **Edit** and **Delete** action buttons, with delete triggering an animated confirmation modal.

11.4 CreateListing & EditListing Pages

Form pages for creating and editing listings. Both feature the **ImageUpload** component for drag-and-drop image handling, with a fallback URL input field. The edit page pre-populates form fields by fetching existing data from the API.

11.5 Login & Register Pages

Authentication forms styled as centered white cards on a gray background. The login page includes "Remember me" functionality with show/hide password toggle. The register page includes password

confirmation with client-side validation. Both use the native `fetch` API and store user data in `localStorage` .

11.6 ImageUpload Component

A reusable drag-and-drop image upload component that reads files as Base64 data URLs via `FileReader` . Supports drag-over visual feedback, click-to-browse, image preview with hover overlay, and a remove button. Used in both Create and Edit listing forms.

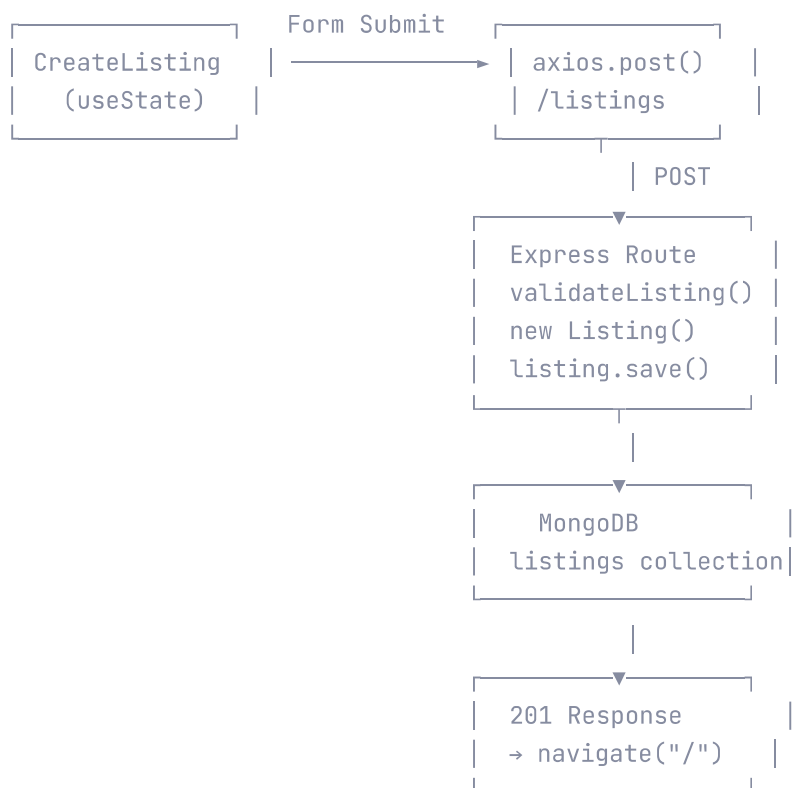
12. Data Flow & State Management

12.1 State Management Approach

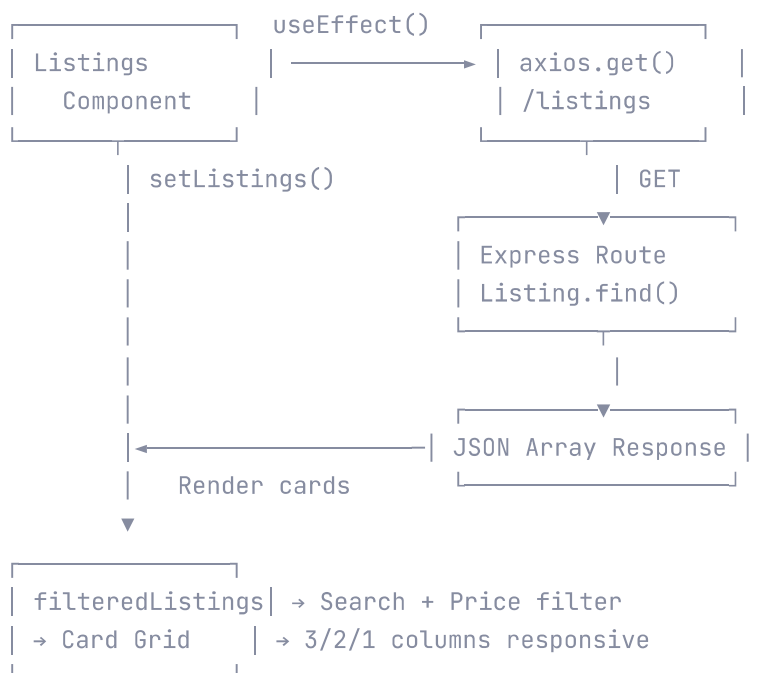
The application uses **React's built-in state management** via the `useState` and `useEffect` hooks. There is no external state management library (like Redux or Zustand). Each component manages its own local state, and authentication state is persisted via `localStorage`.

12.2 Data Flow Diagram

Create Listing Flow:



Browse Listings Flow:



13. UI/UX Design

The application's design is heavily inspired by Airbnb's modern design language, featuring clean typography, spacious layouts, and attention to micro-interactions.

13.1 Design Principles

- **Airbnb-Inspired Aesthetics:** The brand color `#FF385C` , card-based layouts, and category icon bar are directly inspired by Airbnb's design system.
- **Progressive Disclosure:** Information is revealed progressively — listing cards show essential info, detail pages show full info.
- **Skeleton Loading:** Instead of spinners, shimmer-animated skeleton cards provide perceived performance improvement.
- **Responsive Grid:** 3 columns on desktop (>1024px), 2 on tablet (640–1024px), 1 on mobile (<640px).

13.2 CSS Architecture

FILE	LINES	SCOPE
Listings.css	535	Main grid, cards, categories, filter, star ratings
ShowListing.css	320	Detail page, hero image, delete modal
ListingForm.css	276	Create/edit forms, image upload zone
Navbar.css	233	Navigation bar, search, auth links
Hero.css	189	Hero banner, background animation
LoginPage.css	178	Login form card
RegisterPage.css	154	Register form card
SkeletonCard.css	50	Loading skeleton animation
index.css	49	Global resets and base styles

13.3 Animations & Micro-Interactions

- **Card Hover:** Box shadow elevation + 2px translateY lift + image zoom (scale 1.05)
- **Skeleton Shimmer:** Linear gradient sliding left-to-right at 1.4s intervals
- **Delete Modal:** Overlay fadeIn + card slideUp animation
- **Hero Background:** Subtle scale animation (1.05 → 1.0) on hover over 8 seconds
- **Category Active:** Underline transition with brand color on category selection
- **Image Upload:** Drag-over border highlight and opacity change

14. Testing & Validation

14.1 Server-Side Validation

All API inputs are validated using **Joi** schemas before processing. The `validateListing` and `validateReview` middleware functions check incoming request bodies against predefined schemas and return **400 Bad Request** errors with detailed messages when validation fails.

14.2 Client-Side Validation

- **Required Fields:** HTML5 `required` attributes on form inputs
- **Password Match:** Registration form validates that password and confirm password match before submission
- **Price Validation:** Minimum value of 0 enforced via Joi on the server

14.3 Error Handling

- **API Errors:** Custom `ExpressError` class with status codes; global error middleware catches all unhandled errors
- **Async Errors:** `wrapAsync` utility catches promise rejections in async route handlers
- **Frontend Errors:** try-catch blocks in API calls with error state displayed to users
- **404 Handling:** Individual routes return 404 for non-existent resources

15. Future Enhancements



Cloud Image Storage

Integrate Cloudinary or AWS S3 for scalable image storage instead of Base64/URL-based images.



Map Integration

Add Mapbox or Google Maps to show listing locations on interactive maps.



Listing Ownership

Add an owner/author field to listings so only the creator can edit or delete their listings.



Actual Reviews UI

Build a frontend interface for submitting and viewing reviews on listing detail pages.



Enhanced Security

Move to JWT tokens, environment variables for secrets, and remove localStorage password storage.



Booking System

Add a booking flow with date selection, availability calendar, and email confirmations.



Category Filtering

Connect category buttons to actual backend filtering logic for categorized browsing.



Deployment

Deploy frontend to Vercel/Netlify and backend to Render/Railway with MongoDB Atlas.

16. Conclusion

The **Wanderlust** project successfully demonstrates the development of a complete, full-stack web application using the **MERN Stack** (MongoDB, Express.js, React.js, Node.js). The application showcases key web development concepts including:

- **RESTful API Design** — A well-structured Express.js backend with clear route definitions, middleware layers, and JSON responses.
- **Database Modeling** — Proper Mongoose schema design with data types, validation, defaults, and inter-collection references.
- **Authentication & Sessions** — Secure user authentication using Passport.js with session-based persistence in MongoDB.
- **Modern Frontend Architecture** — A React.js SPA with component-based design, hooks-based state management, and client-side routing.
- **Responsive UI/UX** — An Airbnb-inspired design with custom CSS, responsive grid layouts, animations, and skeleton loading states.
- **Input Validation** — Dual-layer validation with Joi on the server and HTML5 validation on the client.
- **Image Handling** — Drag-and-drop image upload with Base64 encoding and fallback URL support.

The project serves as a comprehensive example of modern web application development, covering the complete lifecycle from database design to API implementation to frontend UI construction. With the listed future enhancements, the application can be further expanded into a production-ready platform.

✦ Summary

Wanderlust is a full-stack MERN application with **3 database models**, **9 API endpoints**, **7 frontend routes**, **6 page components**, **5 reusable components**, and **~2,000+ lines of custom CSS** — delivering a polished, Airbnb-inspired property listing experience.

Wanderlust — Major Project Report

Full-Stack Web Application • MERN Stack • React.js + Express.js + MongoDB

Generated for academic documentation purposes